

DEVELOPMENT WORK LOG

AI Web Tester — Stop Test Feature

Full-stack implementation: FastAPI backend & Next.js frontend

Developer: Biswas | Date: 26 May 2026 | Project: C:\Users\biswa\OneDrive\Desktop\AI assistant tool\ai-web-tester

TABLE OF CONTENTS

- 1. Project Overview & Tech Stack
- 2. Work Summary — What Was Done Today
- 3. Issue Encountered — Disk Full & File Corruption
- 4. Backend Changes (main.py, agent.py)
- 5. Frontend Changes (index.js, reports.js, TestResult.js)
- 6. Full File Diff's — Key Code Changes
- 7. How the Stop Test Feature Works
- 8. Testing Notes

1. Project Overview & Tech Stack

The **AI Web Tester** is an automated website QA tool that uses Claude AI (Anthropic) to control a real browser (Playwright) and verify test goals written in plain English. Results and screenshots are stored in a local SQLite database and served via a Next.js frontend.

LAYER	TECHNOLOGY	PURPOSE
AI Model	Claude Haiku 4.5 (claude-haiku-4-5-20251001)	Decides browser actions step-by-step
Browser	Playwright (Python async)	Controls real browser; headed = MS Edge, headless = Chromium
Backend	FastAPI + uvicorn	REST API + SSE streaming on port 8000

Database	SQLite (WAL mode)	Stores test runs, steps, screenshots
Frontend	Next.js + Tailwind CSS	UI on port 3000
Key files	backend/agent.py, main.py, browser.py, database.py	Core logic

2. Work Summary — What Was Done Today

5

FILES MODIFIED

~200

LINES ADDED

1

FEATURE SHIPPED

Today's primary goal was to implement the **Stop Test** feature end-to-end — allowing users to safely halt a running AI browser test at any step and receive a partial report of completed work.

- Backend — Stop Registry & /stop Endpoint (main.py)**
Added `_stop_events` global dict, `POST /stop/{run_id}` endpoint, `run_id` form param on both `/test` and `/test-stream` endpoints, and `asyncio.Event` registration/cleanup logic.
- Backend — Agent Stop Signal (agent.py)**
Added `stop_event: asyncio.Event` parameter to `agent.run()`. At the start of each step loop the agent checks if the event is set and returns a STOPPED partial result.
- Frontend — Stop Test Button & State (index.js)**
Added `runId` + `stopping` state, `stopTest()` function, `crypto.randomUUID()` run ID generation, "init" / "stopped" SSE event handling, and red Stop Test button UI.
- Frontend — STOPPED Verdict Display (reports.js + TestResult.js)**
Added amber STOPPED badge to VerdictBadge, STOPPED counter to stats bar, amber verdict banner, and "Test was stopped by user" info box with partial results note.
- Incident — Disk Full / File Corruption**
Mid-edit the C: drive hit 0 bytes free. `index.js` was corrupted to 1 line. Diagnosed available drives (D: 145 GB free, E: 146 GB free). C: drive recovered 7.6 GB. File fully restored.

3. Incident — Disk Full & File Corruption

ISSUE

While editing `frontend/pages/index.js`, a write operation failed with `ENOSPC: no space left on device`. The partial write corrupted the file to a single blank line.

Disk Space Diagnosis

DRIVE	USED	FREE
C:	~183 GB	0 → recovered 7.6 GB (OneDrive evicted local copies)
D:	1.5 GB	145 GB
E:	0.1 GB	146 GB

Root Cause

The C: drive was completely full. After the error was detected, OneDrive's smart tiering evicted locally-cached files to the cloud and freed ~7.6 GB on C:. This was sufficient to restore the project file.

Resolution

Rewrote the complete `index.js` from context memory with all Stop Test changes applied in one atomic write. No data was lost; all edits were reconstructed from the session context.

RECOMMENDATION

Free at least 10–20 GB on C: to avoid recurrence. Large consumers: `OneDrive` (~10.7 GB local cache), `anaconda3` (~4.6 GB), `Downloads` (~2.7 GB). Consider using Disk Cleanup or Windows Storage Settings to remove temporary files.

4. Backend Changes

backend/main.py

ADDED Global stop event registry at module level:

```
# Registry of active test runs: run_id → asyncio.Event (set to signal stop)
_stop_events: dict[str, asyncio.Event] = {}
```

ADDED POST /stop/{run_id} endpoint:

```
@app.post("/stop/{run_id}")
async def stop_test(run_id: str):
    event = _stop_events.get(run_id)
    if event:
        event.set()
        return {"message": "Stop signal sent"}
    # Test may have already finished – not an error
    return {"message": "Test already completed or not found"}
```

UPDATED Both /test and /test-stream endpoints now accept an optional run_id form field and wire up the stop event:

```
@app.post("/test")
async def run_test(..., run_id: Optional[str] = Form(None)):
    active_run_id = run_id or str(uuid.uuid4())
    stop_event = asyncio.Event()
    _stop_events[active_run_id] = stop_event
    try:
        agent = WebTestingAgent()
        result = await agent.run(..., stop_event=stop_event)
    finally:
        _stop_events.pop(active_run_id, None)
```

ADDED SSE init event so the frontend can confirm the run ID immediately:

```
async def event_generator():
    # Send run_id immediately so the frontend can wire up the stop button
    yield f"data: {json.dumps({'type': 'init', 'run_id': active_run_id})}\n\n"
    ...
```

backend/agent.py

ADDED `stop_event` parameter and per-step check:

```
async def run(self, url, goal, credentials=None, headless=True, slow_mo=0,
               on_log=None, stop_event: asyncio.Event = None) -> dict:
    ...
    for step_num in range(self.max_steps):
        # Check for stop signal before starting each new step
        if stop_event and stop_event.is_set():
            duration = (datetime.now() - start_time).total_seconds()
            await emit("stopped", message="Test stopped by user",
                      total_steps=len(steps), duration=round(duration, 2))
            return self._build_result(url, goal, steps, "STOPPED", start_time, "Stopped by
user")
    ...
```

UPDATED `_build_result()` now maps STOPPED verdict to `status: "stopped"` :

```
"status": "completed" if verdict in ("PASSED", "FAILED") else verdict.lower(),
```

5. Frontend Changes

`frontend/pages/index.js`

ADDED State variables:

```
const [runId, setRunId] = useState(""); // active run's ID
const [stopping, setStopping] = useState(false); // disables Stop button after click
```

ADDED `stopTest()` function:

```
const stopTest = async () => {
  if (!runId || stopping) return;
  setStopping(true);
  try {
    await fetch(`${BACKEND}/stop/${runId}`, { method: "POST" });
  } catch {}
};
```

ADDED Run ID generation before each test (works for both headless and headed modes):

```
const activeRunId = crypto.randomUUID();
setRunId(activeRunId);
setStopping(false);
setLoading(true);
...
formData.append("run_id", activeRunId);
```

UPDATED `handleStreamEvent` handles new `"init"` event (backend confirms run ID):

```
} else if (event.type === "init") {
  setRunId(event.run_id); // confirmation from backend
}
```

UPDATED `formatConsoleLog` renders the `"stopped"` event in amber in the execution console:

```
case "stopped":
  return {
    color: "text-amber-400",
    icon: "■",
    text: `Stopped · ${log.total_steps ?? 0} steps · ${log.duration?.toFixed(1) ?? "0"}s`,
```

```
sub: null,
};
```

ADDED Red Stop Test button — visible only when `loading && runId`, disabled while `stopping` :

```
{loading && runId && (
  <button
    type="button"
    onClick={stopTest}
    disabled={stopping}
    className="w-full mt-2 bg-red-500 hover:bg-red-600 disabled:bg-red-300
      text-white font-semibold py-3 px-6 rounded-xl ..."
  >
    {stopping ? (
      <><SpinnerSVG /> Stopping...</>
    ) : (
      <><span className="w-3 h-3 bg-white rounded-sm inline-block" /> Stop Test</>
    )}
  </button>
)}
```

UPDATED Both `finally` blocks (headless and streamed) reset state:

```
} finally {
  setLoading(false);
  setRunId("");
  setStopping(false);
}
```

`frontend/pages/reports.js`

ADDED STOPPED to `VerdictBadge` styles (amber) and stats bar:

```
const styles = {
  PASSED: "bg-green-100 text-green-700 border-green-300",
  FAILED: "bg-red-100 text-red-700 border-red-300",
  ERROR: "bg-orange-100 text-orange-700 border-orange-300",
  STOPPED: "bg-amber-100 text-amber-700 border-amber-300", // ← new
};

// Stats bar now includes:
{ label: "Stopped", value: stats.stopped, color: "text-amber-500" }
```

`frontend/components/TestResult.js`

ADDED Amber verdict banner color and distinct info box for STOPPED:

```
const verdictColor =  
  final_verdict === "PASSED" ? "bg-green-500" :  
  final_verdict === "FAILED" ? "bg-red-500" :  
  final_verdict === "STOPPED" ? "bg-amber-500" : // ← new  
    "bg-orange-400";  
  
// In summary section:  
{final_verdict === "STOPPED" && (  
  <div className="mt-3 p-3 bg-amber-50 border border-amber-200 rounded-lg">  
    <p className="text-xs text-amber-700 font-semibold">Test was stopped by user</p>  
    <p className="text-xs text-amber-600 mt-1">  
      The partial results below show what was completed before stopping.  
    </p>  
  </div>  
)}  
}}
```


6. How the Stop Test Feature Works

The stop mechanism is built on `asyncio.Event` as a lightweight per-run cancellation token:

STEP	WHO	ACTION
1	Frontend	<code>crypto.randomUUID()</code> generates a run ID before the test starts
2	Frontend	Appends <code>run_id</code> to FormData; SSE receives <code>{"type":"init","run_id":"..."}</code> confirming it
3	Frontend	Stop button appears (red, labelled "Stop Test")
4	User	Clicks Stop Test button
5	Frontend	<code>stopTest()</code> POSTs to <code>/stop/{runId}</code> ; button changes to "Stopping..." spinner
6	Backend /stop	Sets the <code>asyncio.Event</code> for that run ID
7	agent.run() loop	Before the next step, checks <code>stop_event.is_set()</code> — returns partial result with verdict = STOPPED
8	Backend SSE	Emits <code>{"type":"stopped",...}</code> then <code>{"type":"result","data":{"...STOPPED..."}}</code>
9	Frontend	Renders amber STOPPED banner with completed steps; cleans up state

KEY SAFETY PROPERTIES

The agent completes its current browser action before stopping — it never interrupts mid-action. The browser is always closed cleanly via `browser.stop()` in the `finally` block. Partial results (all steps completed so far, screenshots) are saved to the database and shown in the report.

Works in Both Modes

MODE	HOW STOP WORKS
Headed (streaming)	Frontend sends stop request while SSE stream is open. Agent checks the event at each step boundary. Browser window visible to user. Console shows ■ Stopped entry.
Headless (REST)	Frontend sends stop POST while axios request is pending. Agent checks event at next step. Result returns with STOPPED verdict from the original request.

7. Testing Notes

HOW TO TEST THE STOP FEATURE

1. Start backend: `cd backend && python main.py`
2. Start frontend: `cd frontend && npm run dev`
3. Open `http://localhost:3000`
4. Enter any URL and a goal (e.g. "Test all navigation links")
5. Click
- RUN TEST

— the blue "Running Test..." button and red

STOP TEST

button will both appear
6. Click
- STOP TEST

— it should change to "Stopping..." (spinner)
7. Within 1–2 steps, the test stops and shows the amber STOPPED result card
8. View Reports page should show STOPPED in amber in the verdict badge and stats bar

Edge Cases Handled

EDGE CASE	BEHAVIOUR
Stop clicked after test finishes	<code>/stop/{id}</code> returns "already completed or not found" — no error
Stop clicked twice	<code>stopping === true</code> guards prevent duplicate POSTs
Test completes before stop request arrives	PASSED/FAILED result shown normally; stop is no-op
Network error on stop request	<code>catch{}</code> swallows the error; agent times out naturally
0 steps completed when stopped	Empty steps array returned; STOPPED badge shown with 0 steps